FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

BSc Data Science & AI - Semester 2

COMPREHENSIVE REVISION NOTE 1

Introduction to AI & Search Algorithms

TOPICS COVERED IN THIS REVISION NOTE

1. Course Introduction & Learning Objectives
2. Definition and Characteristics of Artificial Intelligence
3. Types of AI (Narrow, General, Super AI)
4. History of AI - From 1950s to Present
5. Intelligent Agents and Their Environment
6. Search Problems and Graph Representation
7. Breadth-First Search (BFS) - Complete Algorithm
8. Depth-First Search (DFS) - Complete Algorithm
9. Uniform Cost Search (UCS) - Weighted Graphs
10. A* Search Algorithm - Heuristic Search

SECTION 1: UNDERSTANDING ARTIFICIAL INTELLIGENCE

1.1 Definition of Artificial Intelligence

Artificial Intelligence (AI) is a branch of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence. The fundamental goal of AI is to develop machines that can **simulate human cognitive processes** such as learning from experience, adapting to new situations, understanding language, recognizing patterns, and making decisions.

The concept of artificial intelligence emerged from a fundamental question that has fascinated scientists and philosophers for centuries: *"What is intelligence, and can we recreate it artificially?"* This question drives the entire field of AI research and development. When we talk about AI, we are essentially discussing the science of making machines behave in ways that we would consider intelligent if observed in humans.

AI systems are designed to perceive their environment, process information, reason about that information, and take actions to achieve specific goals. Unlike traditional computer programs that follow rigid, pre-defined instructions, AI systems can learn from data, identify patterns, and improve their performance over time without being explicitly programmed for every possible scenario.

■ FIVE KEY PROCESSES THAT AI SIMULATES

1. Learning: The ability to acquire new knowledge and improve performance based on experience and data. Machine learning algorithms enable AI systems to learn patterns from large datasets and make predictions or decisions without being explicitly programmed.

2. Reasoning: The capacity to use rules, logic, and inference to draw conclusions and make decisions. AI systems can apply logical rules to known facts to derive new information or solve problems systematically.

3. Problem Solving: The ability to analyze complex situations, identify potential solutions, and select the most appropriate course of action. This involves search algorithms, optimization techniques, and strategic planning.

4. Perception: The ability to interpret and understand sensory inputs such as images, sounds, and text. Computer vision, speech recognition, and natural language processing are examples of AI perception capabilities.

5. Language Understanding: The ability to comprehend, interpret, and generate human language in a meaningful way. This enables AI systems to communicate naturally with humans and process textual information.

1.2 Real-World Applications of AI

Artificial Intelligence has become an integral part of our daily lives, often working behind the scenes to enhance our experiences and solve complex problems. Understanding these applications helps us appreciate the practical significance of AI concepts we study in this course.

AI Assistants (Alexa, Siri, Google Assistant)

Virtual assistants represent one of the most visible applications of AI in consumer technology. These systems combine multiple AI capabilities including speech recognition (converting spoken words to text), natural language processing (understanding the meaning and intent behind words), knowledge retrieval (finding relevant information), and speech synthesis (generating natural-sounding responses). When you ask Alexa to play a song or Siri to set a reminder, these systems must understand your request, process it, and take appropriate action—all in real-time.

Autonomous Vehicles (Tesla Autopilot, Self-Driving Cars)

Self-driving cars represent one of the most complex applications of AI, requiring the integration of multiple AI subsystems. These vehicles use computer vision to interpret camera feeds, sensor fusion to combine data from cameras, radar, and lidar, path planning algorithms to determine safe routes, and real-time decision-making to respond to changing road conditions. The AI must continuously perceive its environment, predict the behavior of other vehicles and pedestrians, and make split-second decisions to ensure safety.

Recommendation Systems (YouTube, Netflix, Amazon)

Every time you see personalized suggestions on streaming platforms or e-commerce sites, you're interacting with AI-powered recommendation systems. These systems analyze your past behavior (what you've watched, purchased, or clicked on), compare it with patterns from millions of other users, and use machine learning algorithms to predict what you might enjoy next. The goal is to increase user engagement by surfacing relevant content from vast libraries of options.

Fraud Detection in Banking

Financial institutions use AI to protect customers from fraudulent transactions. These systems analyze patterns in transaction data—including amount, location, time, merchant type, and user behavior—to identify anomalies that might indicate fraud. Machine learning models are trained on historical data of both legitimate and fraudulent transactions, enabling them to flag suspicious activity in real-time while minimizing false positives.

1.3 Key Characteristics of AI Systems

For a system to be considered truly intelligent, it must exhibit certain fundamental characteristics. These characteristics distinguish AI systems from traditional software programs and enable them to handle complex, real-world situations.

Characteristic	Detailed Explanation
Adaptability	The ability to adjust behavior based on changes in the environment or new data. An adaptive AI system can modify its strategies when circumstances change, rather than failing when faced with unexpected situations.
Problem Solving	The capability to analyze problems, generate potential solutions, evaluate alternatives, and select optimal approaches. This involves both systematic search through solution spaces and creative reasoning.
Self-Correction	The ability to recognize errors, learn from mistakes, and improve future performance. Self-correcting systems can identify when their predictions or actions were wrong and adjust their models accordingly.
Interaction	The capacity to communicate naturally with humans through various modalities including speech, text, and gestures. Good interaction design makes AI systems accessible and useful to non-technical users.
Autonomy	The ability to operate independently without constant human supervision. Autonomous systems can perceive their environment, make decisions, and take actions to achieve goals with minimal human intervention.

1.4 Classification of Artificial Intelligence

AI systems can be classified in multiple ways based on their capabilities, functionality, and the scope of problems they can solve. Understanding these classifications helps us appreciate where current AI technology stands and what future developments might bring.

Classification Based on Capability

Narrow AI (Weak AI) - Current State of AI

Narrow AI refers to artificial intelligence systems that are designed and trained for a specific task or a limited range of tasks. These systems can perform their designated tasks exceptionally well—often surpassing human performance—but cannot generalize their abilities to other domains. For example, an AI system trained to play chess cannot suddenly play Go or diagnose diseases. All AI systems in use today, including ChatGPT, image recognition systems, and recommendation engines, fall under the category of Narrow AI. While these systems appear intelligent within their domains, they lack the general reasoning abilities that humans possess.

General AI (Strong AI) - The Research Goal

General AI represents the goal of creating machines with human-level intelligence across all cognitive domains. A General AI system would be able to learn any intellectual task that a human can learn, transfer knowledge between domains, reason abstractly, and adapt to entirely new situations without specific training. This type of AI remains largely theoretical and is the subject of ongoing research. The challenge lies not just in creating systems that can perform multiple tasks, but in developing systems that can truly understand, reason, and learn in the flexible way humans do.

Super AI (Artificial Superintelligence) - Hypothetical Future

Super AI refers to a hypothetical future AI that would surpass human intelligence in virtually every field, including scientific creativity, general wisdom, and social skills. This concept remains purely speculative and raises significant philosophical and ethical questions. If such an AI were ever developed, it could potentially solve complex global challenges but also poses existential risks that are actively debated by researchers, ethicists, and policymakers.

Type	Capabilities	Examples & Status
Narrow AI	Specialized in specific tasks; cannot generalize to other domains	Siri, Chess engines, Image classifiers. **STATUS: Current AI**
General AI	Human-level intelligence; can learn and perform any intellectual task	No examples exist yet. **STATUS: In Research**
Super AI	Exceeds human intelligence in all domains; self-improving	Purely theoretical. **STATUS: Hypothetical**

Classification Based on Functionality

1. Reactive Machines: The simplest type of AI that can only react to current situations based on pre-defined rules. These systems have no memory of past events and cannot learn from experience. IBM's Deep Blue chess computer is a classic example—it could evaluate millions of chess positions but had no memory of previous games.

2. Limited Memory AI: These systems can use past experiences to inform future decisions. They can learn from historical data and improve over time. Most modern AI applications fall into this category, including self-driving cars that remember recent observations and ChatGPT that maintains conversation context.

3. Theory of Mind AI: This represents future AI that could understand human emotions, beliefs, intentions, and thought processes. Such AI would be able to engage in truly social interactions, understanding not just what people say but what they mean and feel. This remains an active research area.

4. Self-Aware AI: The most advanced hypothetical form of AI that would possess consciousness and self-awareness. Such an AI would understand its own existence, have subjective experiences, and potentially have desires and

emotions. This raises profound philosophical questions about the nature of consciousness.

SECTION 2: THE HISTORY OF ARTIFICIAL INTELLIGENCE

Understanding the history of AI helps us appreciate how the field has evolved, why certain approaches succeeded or failed, and what lessons we can apply to current research. The development of AI has been marked by periods of great optimism and funding, followed by periods of disappointment and reduced support—patterns that continue to shape the field today.

2.1 The Birth of AI (1940s-1950s)

The conceptual foundations of AI began in the 1940s and 1950s with the pioneering work of mathematicians and computer scientists who dared to ask whether machines could think. The development of electronic computers during World War II created the technological foundation for these ideas to be explored practically.

Alan Turing and the Turing Test (1950): In 1950, British mathematician Alan Turing published his landmark paper "Computing Machinery and Intelligence," which posed the question "Can machines think?" Rather than attempting to define thinking, Turing proposed a practical test: if a machine could engage in a conversation indistinguishable from a human, it could be considered intelligent. This "Imitation Game" became known as the Turing Test and remains an influential concept in AI research, though its limitations are now well understood.

The Dartmouth Conference (1956): The field of AI was officially born at a summer workshop held at Dartmouth College, organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. McCarthy coined the term "Artificial Intelligence" for this conference. The proposal optimistically stated that "every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it." Participants believed that significant progress could be made in just one summer. While this timeline proved wildly optimistic, the conference established AI as a distinct field of research.

> **■■ IMPORTANT CONCEPT: AI WINTER**
>
> **Definition:** An AI Winter refers to a period of reduced funding, interest, and research activity in artificial intelligence due to unmet expectations and disappointment with progress.
>
> **First AI Winter (1970s):** By the early 1970s, it became clear that the optimistic predictions of the 1950s and 1960s were unrealistic. Researchers discovered that many problems were far more difficult than anticipated. The limited computational power of the time could not support the complex reasoning required for general intelligence. Government funding agencies, disappointed by the lack of progress, significantly reduced AI research funding.
>
> **Key Lesson:** Over-promising and under-delivering can damage an entire field. Understanding this history helps us maintain realistic expectations about AI capabilities and limitations.

2.2 Key Milestones in AI History

Year	Milestone Event
1950	Alan Turing publishes 'Computing Machinery and Intelligence' - proposes the Turing Test
1956	Dartmouth Conference - Term 'Artificial Intelligence' is coined; AI established as field
1970s	**First AI Winter** - Funding cuts due to unmet expectations and hardware limitations
1980s	Expert Systems revival - Rule-based AI for medical diagnosis, engineering applications
1997	**IBM Deep Blue** defeats world chess champion Garry Kasparov

2012	**AlexNet** wins ImageNet - Deep learning breakthrough in computer vision
2016	**AlphaGo** defeats world Go champion Lee Sedol - Breakthrough in game AI
2020s	**Generative AI Era** - ChatGPT, DALL-E, and large language models transform AI applications

2.3 Why AI Emerged Strongly in Recent Times

The remarkable progress in AI over the past decade can be attributed to the convergence of several factors:

• **Big Data Availability:** The explosion of digital data from the internet, sensors, and connected devices provides the vast training datasets that modern AI systems require.

• **Advanced Hardware (GPUs/TPUs):** Graphics Processing Units, originally designed for video games, proved ideal for the parallel computations required by neural networks.

• **Algorithm Improvements:** Researchers developed better neural network architectures (CNNs, RNNs, Transformers), training techniques, and optimization methods.

• **Cloud Computing:** Cloud platforms made powerful computing resources accessible without massive upfront investments.

• **Open Source Movement:** Frameworks like TensorFlow, PyTorch, and scikit-learn made it easy to implement AI algorithms.

SECTION 3: INTELLIGENT AGENTS

The concept of an intelligent agent is fundamental to understanding AI. An agent is anything that can perceive its environment through sensors and act upon that environment through actuators. An intelligent agent is one that acts rationally to achieve its goals given its knowledge and perceptions.

3.1 Agent-Environment Interaction

Every intelligent agent operates within an environment and follows a cycle of perception, reasoning, and action. Think of yourself as an intelligent agent: you perceive your environment through your senses (eyes, ears, touch), your brain processes this information and makes decisions, and your body executes actions (walking, speaking, manipulating objects).

■ THE AGENT-ENVIRONMENT CYCLE

1. PERCEPTION: The agent receives inputs from the environment through sensors. For a robot, this might be camera images; for a chess program, it's the current board state.

2. REPRESENTATION: The agent creates an internal model or representation of the environment. This mental map allows the agent to reason about the world.

3. REASONING/PLANNING: Using its internal representation, the agent reasons about what actions to take. This involves search algorithms, logic, and decision-making processes.

4. ACTION/EXECUTION: The agent performs actions in the environment through actuators. The cycle then repeats as the agent perceives the results of its actions.

3.2 Types of Agents

Simple Reflex Agents: These agents select actions based solely on the current percept, ignoring history. They work in fully observable environments with simple condition-action rules. Example: A thermostat that turns on heating when temperature drops below a threshold.

Model-Based Reflex Agents: These agents maintain an internal state that tracks aspects of the world not directly observable. They can handle partially observable environments by using their model to fill in missing information.

Goal-Based Agents: These agents have explicit goals and select actions that move them toward achieving these goals. They can perform search and planning to find sequences of actions that lead to the goal state.

Utility-Based Agents: These agents have a utility function that measures how desirable different states are. They can make trade-offs between conflicting goals and handle uncertainty by maximizing expected utility.

Learning Agents: These agents can improve their performance over time by learning from experience. They have components for learning (improving based on feedback), performance (selecting actions), and critique (evaluating outcomes).

3.3 Environment Types

Environment Type	Description & Examples
Fully Observable	Agent has complete access to the environment state. Examples: Chess (entire board visible), Tic-tac-toe

Partially Observable	Agent can only see part of the environment. Examples: Poker (hidden cards), Self-driving car (limited sensor range)
Deterministic	Same action in same state always produces same result. Examples: Calculator, Crossword puzzle
Stochastic	Outcomes involve randomness/uncertainty. Examples: Weather prediction, Stock market trading

SECTION 4: SEARCH PROBLEMS IN AI

Search is one of the most fundamental techniques in AI. Many problems can be formulated as search problems: finding a path from an initial state to a goal state. Whether it's finding the shortest route on a map, solving a puzzle, or planning a sequence of actions for a robot, search algorithms provide systematic ways to explore possibilities and find solutions.

4.1 Components of a Search Problem

Every search problem can be formally defined using the following components:

- **Initial State (S):** The starting configuration of the problem. This is where the agent begins its search.
- **Goal State (G):** The desired end configuration that represents a solution. There may be multiple goal states.
- **Actions/Operators:** The possible moves or transitions available from each state.
- **Transition Model:** A description of what each action does—how it transforms one state into another.
- **Path Cost:** A function that assigns a numeric cost to each path, allowing comparison of solutions.
- **Solution:** A sequence of actions leading from the initial state to a goal state.

4.2 Real-World Example: Flight Booking

Let's understand search problems through a practical example. Suppose you want to travel from Jodhpur to Kolkata by air, but there's no direct flight. You need to find a route through connecting cities.

Component	In Our Example
Initial State	Jodhpur (your starting location)
Goal State	Kolkata (your destination)
States	Cities: Jodhpur, Ahmedabad, Delhi, Jaipur, Kolkata, etc.
Actions	Taking a flight from one city to another
Path Cost	Flight fares (or travel time, or number of stops)
Solution	A sequence like: Jodhpur → Delhi → Kolkata

4.3 Graph Representation of Search Problems

Search problems are naturally represented as graphs, which provide a powerful framework for visualization and algorithmic solution. In graph representation:

- **Nodes (Vertices):** Represent states. Each possible configuration of the problem is a node.
- **Edges:** Represent actions. An edge from node A to node B means there's an action that transforms state A into state B.
- **Directed Edges:** The direction matters—just because you can go from A to B doesn't mean you can go from B to A.
- **Edge Weights:** Represent the cost of actions. Different edges can have different weights (costs).

4.4 Types of Search Algorithms

Uninformed (Blind) Search	Informed (Heuristic) Search

Definition: Search strategies that have no additional information about states beyond the problem definition. **Characteristics:** • No knowledge of how close a state is to the goal • Can only distinguish goal from non-goal states • May explore many unnecessary states **Algorithms:** • Breadth-First Search (BFS) • Depth-First Search (DFS)	**Definition:** Search strategies that use problem-specific knowledge to guide the search toward the goal. **Characteristics:** • Uses heuristics to estimate distance to goal • Can prioritize promising paths • Generally more efficient than blind search **Algorithms:** • Uniform Cost Search (UCS) • A* Search Algorithm

SECTION 5: BREADTH-FIRST SEARCH (BFS)

Breadth-First Search is one of the simplest and most intuitive search algorithms. It systematically explores nodes level by level, examining all nodes at depth d before moving to depth d+1. This systematic approach guarantees finding the shortest path (in terms of number of edges) if one exists.

5.1 How BFS Works

Imagine you're looking for someone in a building. BFS strategy would be: first check everyone on your floor, then everyone on adjacent floors, then floors further away. You expand outward in all directions equally before going deeper in any single direction.

BFS uses a **queue (First-In-First-Out)** data structure to track nodes to visit. Nodes discovered first are explored first, ensuring level-by-level exploration.

■ BFS ALGORITHM - STEP BY STEP

Step 1: Initialize - Create an empty queue and add the initial state. Mark the initial state as visited.

Step 2: Check if queue is empty - If yes, no solution exists. Return failure.

Step 3: Dequeue - Remove the front node from the queue. This becomes the current node.

Step 4: Goal Test - Check if current node is a goal state. If yes, return the path to this node as the solution.

Step 5: Expand - Find all neighbors (adjacent nodes) of the current node.

Step 6: Enqueue unvisited - For each neighbor not yet visited, mark it as visited and add it to the queue.

Step 7: Repeat - Go back to Step 2.

5.2 Worked Example: BFS Traversal

Consider a graph with: Initial state S, Goal states G1, G2, G3, and intermediate states A, B, C, D, E, F. Edges: S→A, S→B, S→D, A→B, A→G1, B→A, B→C, D→S, D→C, D→E, C→F, C→G2, E→G3, F→D, F→G3.

Tracing BFS:

Level 0: Start at S. Queue: [S]. Visited: {S}

Level 1: Expand S → neighbors are A, B, D. Queue: [A, B, D]. Visited: {S, A, B, D}

Level 2: Expand A → neighbors B (visited), G1. Queue: [B, D, G1]. Check: Is G1 a goal? YES!

Solution Path: S → A → G1 (Length: 2 edges)

5.3 Properties of BFS

- **Completeness:** BFS is complete - if a solution exists, BFS will find it (assuming finite branching factor).
- **Optimality:** BFS finds the shortest path in terms of number of edges (not considering weights).
- **Time Complexity:** O(b^d) where b is branching factor and d is depth of solution.
- **Space Complexity:** O(b^d) - must store all nodes at the frontier level.

■■ CRITICAL: AVOIDING INFINITE LOOPS

In graphs with cycles, an algorithm could visit the same nodes repeatedly, creating an infinite loop. To prevent this, BFS maintains a **visited set** that tracks all nodes already explored.

Rule: Before adding any node to the queue, check if it's already in the visited set. If yes, skip it (it's a dead end in terms of exploration). This ensures each node is processed at most once.

SECTION 6: DEPTH-FIRST SEARCH (DFS)

Depth-First Search takes the opposite approach to BFS. Instead of exploring all nodes at one level before moving deeper, DFS explores as deep as possible along each branch before backtracking. It's like exploring a maze by always taking the first available turn and only backing up when you hit a dead end.

6.1 How DFS Works

DFS uses a **stack (Last-In-First-Out)** data structure, or equivalently, recursion. When exploring, it goes as deep as possible before considering alternatives.

Backtracking: When DFS reaches a node with no unvisited neighbors (a dead end), it backtracks to the most recent node that has unexplored neighbors and continues from there. This systematic exploration ensures all reachable nodes are eventually visited.

■ DFS ALGORITHM - STEP BY STEP

Step 1: Initialize - Create an empty stack and push the initial state. Mark it as visited.

Step 2: Check if stack is empty - If yes, no solution exists. Return failure.

Step 3: Pop - Remove the top node from the stack. This becomes the current node.

Step 4: Goal Test - Check if current node is a goal state. If yes, return the path.

Step 5: Expand - Find all unvisited neighbors of the current node.

Step 6: Push neighbors - Add unvisited neighbors to the stack and mark them as visited in current path.

Step 7: Repeat - Go back to Step 2.

6.2 Worked Example: DFS Traversal

Using the same graph as BFS, let's trace DFS (going alphabetically when multiple choices exist):

Start: S. Stack: [S]. Visited: {S}

Pop S: Neighbors A, B, D. Push all. Stack: [A, B, D]. Visited: {S, A, B, D}

Pop D: Neighbors C, E. Push both. Stack: [A, B, C, E]

Pop E: Neighbor G3. Push G3. Stack: [A, B, C, G3]

Pop G3: Is G3 a goal? YES!

Solution Path: S → D → E → G3 (Length: 3 edges)

Notice: DFS found G3, not G1 which might have been closer via a different path! DFS does NOT guarantee the shortest path.

6.3 BFS vs DFS: Detailed Comparison

Property	BFS	DFS
Exploration	Level by level (breadth-first)	Branch by branch (depth-first)
Data Structure	Queue (FIFO)	Stack (LIFO) or Recursion

Shortest Path	**YES ✓** (in edges)	**NO ✗**
Memory Usage	Higher - stores entire frontier	Lower - stores current path only
Best For	Shortest path problems, shallow goals	Memory-limited situations, deep goals

SECTION 7: UNIFORM COST SEARCH (UCS)

While BFS finds the shortest path in terms of number of edges, many real-world problems have different costs for different actions. Uniform Cost Search addresses this by always expanding the node with the **lowest cumulative path cost** from the start.

7.1 The Need for UCS

Consider our flight booking example. A route with fewer stops isn't necessarily cheaper or faster. A direct flight might cost ■15,000, while a route with two stops might cost only ■8,000. BFS would prefer the direct flight (fewer edges), but UCS would correctly identify the cheaper option by considering actual costs.

UCS COST FUNCTION

g(n) = Total path cost from start to node n

UCS always expands the node with the smallest g(n) value

7.2 UCS Algorithm

UCS is similar to BFS but uses a **priority queue** ordered by path cost instead of a regular queue. Nodes with lower cumulative cost are expanded first.

- Initialize priority queue with start node (cost = 0)
- Pop node with lowest g(n) from queue
- If it's a goal, return the solution
- For each neighbor, calculate cumulative cost: g(neighbor) = g(current) + edge_cost
- Add neighbors to queue (or update if better path found)
- Repeat until goal found or queue empty

7.3 Worked Example: UCS

Given weighted graph: S→A(5), S→B(9), S→D(6), A→B(3), A→G1(9), B→C(1), D→C(2), D→E(3), C→G2(5), E→G3(7)

Iteration 1: Priority Queue: [(S, 0)]. Pop S. Expand to A(5), B(9), D(6). PQ: [(A,5), (D,6), (B,9)]

Iteration 2: Pop A (cost 5). Expand to B(5+3=8), G1(5+9=14). PQ: [(D,6), (B,8), (B,9), (G1,14)]

Iteration 3: Pop D (cost 6). Expand to C(6+2=8), E(6+3=9). PQ: [(C,8), (B,8), (E,9), (B,9), (G1,14)]

Iteration 4: Pop C (cost 8). Expand to G2(8+5=13). PQ: [(B,8), (E,9), (B,9), (G2,13), (G1,14)]

Continue... Eventually G2 is popped with cost 13.

Solution: S → D → C → G2 with total cost 13

SECTION 8: A* SEARCH ALGORITHM

A* (pronounced 'A-star') is one of the most important and widely-used search algorithms in AI. It combines the best features of UCS (guaranteed optimality) with heuristic guidance (efficiency). A* is used in GPS navigation, video game pathfinding, robotics, and many other applications.

8.1 The A* Innovation

While UCS considers only the cost already incurred (past), A* also considers an estimate of remaining cost (future). This allows A* to intelligently prioritize nodes that appear to be on the most promising path to the goal.

THE A* EVALUATION FUNCTION

$$f(n) = g(n) + h(n)$$

Where:
- **f(n)** = Total estimated cost of path through n
- **g(n)** = Actual cost from start to n (known exactly)
- **h(n)** = Heuristic estimate from n to goal (estimated)

8.2 The Heuristic Function h(n)

The heuristic function h(n) provides an estimate of the cost from node n to the nearest goal. This estimate uses problem-specific knowledge. For example, in a map navigation problem, h(n) might be the straight-line distance to the destination (which is always ≤ actual road distance).

■■ CRITICAL CONCEPT: ADMISSIBLE HEURISTIC

Definition: A heuristic h(n) is admissible if it **NEVER OVERESTIMATES** the true cost to reach the goal.

Mathematical Condition: $h(n) \leq h^*(n)$ for all nodes n, where $h^*(n)$ is the true optimal cost to the goal.

Why It Matters: If h(n) is admissible, A* is guaranteed to find the optimal solution. If h(n) overestimates, A* might skip the optimal path thinking it's too expensive.

Example: Straight-line distance is an admissible heuristic for road navigation because you can never drive to a destination faster than flying there in a straight line.

8.3 A* vs UCS Comparison

Property	UCS	A*
Evaluation	f(n) = g(n)	f(n) = g(n) + h(n)
Uses Heuristic	No	Yes
Efficiency	Explores in all directions	Guided toward goal
Optimal?	Yes (always)	Yes (if h admissible)

SECTION 9: SAMPLE QUIZ QUESTIONS

9.1 Multiple Choice Questions

1. The term 'Artificial Intelligence' was first coined at which event?

a) Turing Conference 1950
b) Dartmouth Conference 1956
c) MIT AI Lab founding 1959
d) Stanford Conference 1965
Answer: b) Dartmouth Conference 1956

2. Which type of AI can perform any intellectual task that a human can?

a) Narrow AI
b) General AI
c) Super AI
d) Reactive AI
Answer: b) General AI

3. BFS uses which data structure?

a) Stack
b) Queue
c) Priority Queue
d) Linked List
Answer: b) Queue (FIFO)

4. The A* algorithm evaluation function is:

a) f(n) = g(n)
b) f(n) = h(n)
c) f(n) = g(n) + h(n)
d) f(n) = g(n) - h(n)
Answer: c) f(n) = g(n) + h(n)

5. An admissible heuristic:

a) Always overestimates the true cost
b) Never overestimates the true cost
c) Is always equal to the true cost
d) Is always zero
Answer: b) Never overestimates the true cost

6. Which algorithm does NOT guarantee finding the shortest path?

a) BFS
b) DFS
c) UCS
d) A*
Answer: b) DFS

7. IBM Deep Blue defeating Garry Kasparov happened in:

a) 1985
b) 1997
c) 2005
d) 2012
Answer: b) 1997

8. Which is NOT a characteristic of intelligent agents?

a) Perception

b) Reasoning
c) Immortality
d) Learning
Answer: c) Immortality

9.2 True/False Questions

1. The Turing Test was proposed by Alan Turing in 1950. **(TRUE)**

2. DFS always finds the optimal solution. **(FALSE - DFS may find a suboptimal path)**

3. BFS uses more memory than DFS in general. **(TRUE - BFS stores entire frontier)**

4. General AI systems are widely available today. **(FALSE - Still in research)**

5. A* with h(n)=0 behaves exactly like UCS. **(TRUE - f(n)=g(n)+0=g(n))**

6. The AI Winter refers to periods of reduced AI funding. **(TRUE)**

7. UCS is an informed search algorithm. **(FALSE - UCS is uninformed; it doesn't use heuristics)**

8. Reactive machines can learn from past experiences. **(FALSE - No memory)**

SECTION 10: KEY FORMULAS & QUICK REFERENCE

ESSENTIAL FORMULAS TO REMEMBER

A* Evaluation Function:
$f(n) = g(n) + h(n)$

UCS Cost Function:
$f(n) = g(n)$ (only actual cost)

Admissibility Condition:
$h(n) \leq h^*(n)$ for all n
(h(n) must never overestimate true cost h*(n))

Path Cost:
Total Cost = Σ(edge weights on path)

Algorithm Selection Guide

- **Need shortest path (by edges)?** → BFS
- **Need any path with limited memory?** → DFS
- **Need least-cost path (no heuristic)?** → UCS
- **Need optimal path with good heuristic?** → A* (most efficient)

Key Dates to Remember

- **1950:** Turing Test proposed
- **1956:** Dartmouth Conference - AI term coined
- **1970s:** First AI Winter
- **1997:** Deep Blue defeats Kasparov
- **2012:** AlexNet - Deep learning breakthrough
- **2016:** AlphaGo defeats Lee Sedol

— End of Revision Note 1 —
Total Pages: 15+ | Topics: 10 | Questions: 16+
Best of luck with your exam preparation!